

Programación Orientada a objetos

Docente:
Alex Matallana Porras

Alumno:
Maritza Pachón Gómez

Materia:
Estructura de datos

Zipaquirá, Cundinamarca
Corporación Universitaria Minuto de Dios
19 marzo 2025

Tabla de contenido

Objetivos.....	3
Introducción	4
1. ¿Qué es programación orientada a objetos?	5
2. ¿Que son las clases en java?	5
3. Que son los diagramas de clases	5
a. Estructura de los diagramas de clases	5
□ Clases.....	5
□ Atributos	6
□ Operaciones	6
□ Asociaciones.....	6
□ Agregación	6
□ Estereotipos.....	6
□ Diagramas de estructura compuesta	6
b. relación entre ellos y que herramientas gratis se pueden utilizar para construir diagramas de clases	6
□ Asociación (↔)	6
□ Agregación (◊—).....	6
□ Composición (◆—)	7
□ Herencia (Generalización) (▲—)	7
□ Dependencia (↗)	7
□ Realización (⊨)	7
HERRAMIENTAS PARA HACER DIAGRAMAS DE CLASES	7
□ Draw.io	7
□ PlantUML.....	7
□ StarUML	7
□ Lucidchart.....	7
□ UMLet.....	7
□ Modelio	7
4. tipos de métodos en java	7
□ Métodos sin retorno:.....	8
□ Métodos con retorno:.....	8
□ Métodos estáticos:	8
5. Los principios fundamentales de la programación orientada a objetos (POO) son:.....	8
□ Abstracción:	8

□ Encapsulación:	8
□ Herencia:	8
□ Polimorfismo:	9
6. ¿que son las clases abstractas?	9
7. que son interfaces en java y como se construyen	9
Conclusiones	10
Referencias Bibliograficas.....	10

Objetivos

- ❖ Comprender los fundamentos de la Programación Orientada a Objetos (POO) en Java, incluyendo sus principios, estructura y elementos clave.

Objetivos Específicos

- ❖ Explicar el concepto de Programación Orientada a Objetos y su importancia en el desarrollo de software.
- ❖ Describir la estructura y funcionalidad de las clases en Java, incluyendo atributos, métodos y constructores.
- ❖ Analizar los diagramas de clases UML y su utilidad en la representación de la estructura del software.
- ❖ Identificar los diferentes tipos de métodos en Java y su aplicación en el desarrollo de programas.
- ❖ Explicar los principios fundamentales de la POO: abstracción, encapsulación, herencia y polimorfismo.

Introducción

La Programación Orientada a Objetos (POO) es un paradigma esencial en el desarrollo moderno de software, permitiendo la creación de sistemas modulares, reutilizables y fáciles de mantener. A través de conceptos como clases, objetos, herencia y polimorfismo, este enfoque ofrece una manera intuitiva de modelar problemas del mundo real en código. Java, como uno de los lenguajes más utilizados en el desarrollo de software, implementa la POO de manera robusta, permitiendo a los desarrolladores diseñar aplicaciones eficientes y escalables. En este documento, exploraremos los conceptos fundamentales de la POO en Java, la estructura de las clases, los diagramas UML, los diferentes tipos de métodos y la relación entre clases abstractas e interfaces. Además, se presentarán herramientas útiles para la representación gráfica de la arquitectura de software mediante diagramas de clase

1. ¿Qué es programación orientada a objetos?

- ❖ La Programación Orientada a Objetos (POO) es un paradigma de desarrollo que organiza el software en clases y objetos, facilitando la reutilización y el mantenimiento del código. A diferencia de los enfoques anteriores, que priorizaban la lógica, la POO se centra en la interacción entre objetos, lo que mejora la estructura y escalabilidad de los sistemas. Lenguajes como Java, C# y Python han impulsado su uso, permitiendo crear aplicaciones más flexibles y modulares mediante la definición de clases como plantillas para generar objetos que colaboran entre sí.

2. ¿Que son las clases en java?

- ❖ En Java, una clase es una plantilla para crear objetos y constituye la base de la Programación Orientada a Objetos (POO). Dentro de una clase, se definen los datos (atributos) y el código (métodos) que actúan sobre esos datos.
- ❖ Las clases permiten organizar el software en estructuras modulares y reutilizables, facilitando el mantenimiento y escalabilidad de los programas. Un objeto es una instancia de una clase, y su comportamiento se define a través de métodos.
- ❖ Comprender clases, objetos y métodos es esencial para desarrollar aplicaciones en Java, ya que estos conceptos permiten modelar elementos del mundo real y construir programas más eficientes y estructurados.

3. Que son los diagramas de clases

- ❖ Los diagramas de clases son un tipo de diagrama UML que representa la estructura de un sistema orientado a objetos. Muestran las clases, sus atributos, métodos y las relaciones entre ellas, como herencia, asociación y composición.
- ❖ Estos diagramas ayudan a visualizar la arquitectura del software, facilitando su diseño, mantenimiento y comprensión. Son ampliamente utilizados en el desarrollo con Java, C#, Python, entre otros, permitiendo una mejor planificación antes de la implementación del código.

a. Estructura de los diagramas de clases

- ❖ Los diagramas de clases están compuestos por clases, atributos, operaciones, asociaciones, generalizaciones, estereotipos y objetos.
- ❖ Clases
 - i. Se representan como rectángulos
 - ii. Cada clase tiene un nombre, atributos y operaciones

- iii. El nombre de la clase es obligatorio
- iv. Las clases pueden agruparse en paquetes
- ❖ Atributos
 - v. Determinan las características de la clase
 - vi. Las propiedades son un tipo específico de atributo
- ❖ Operaciones
 - vii. Representan las acciones posibles en la clase
 - viii. También se les conoce como métodos
- ❖ Asociaciones
 - ix. Las relaciones entre clases se muestran con líneas y puntas de flecha
 - x. Los tipos de asociación son bidireccional, unidireccional, agregación y reflexiva
- ❖ Agregación
 - xi. Representa una relación de "parte de"
 - xii. Por ejemplo, la clase 2 es parte de la clase 1
- ❖ Estereotipos
 - xiii. Se sitúan sobre el nombre de la clase y entre el símbolo: << >>
- ❖ Diagramas de estructura compuesta
 - xiv. Desglosan la red de clases, interfaces y componentes
 - xv. Muestra cómo interactúan estos elementos entre sí

Los diagramas de clases son los más comunes en el modelado de sistemas orientados a objetos.

b. relación entre ellos y que herramientas gratis se pueden utilizar para construir diagramas de clases

- ❖ En los diagramas de clases UML, las relaciones entre clases representan cómo interactúan o están conectadas. Estas son las principales relaciones que puedes utilizar:
- ❖ Asociación (↔)
 - i. Representa una relación entre dos clases.
 - ii. Puede ser unidireccional o bidireccional.
 - iii. Ejemplo: Un Alumno está asociado a una Escuela.
- ❖ Agregación (◊—)
 - iv. Relación "tiene un" donde una clase contiene a otra, pero los objetos pueden existir por separado.
 - v. Ejemplo: Un Carro tiene varias Ruedas, pero las ruedas pueden existir sin el carro.

- ❖ Composición (◆—)
 - vi. Relación "tiene un" más fuerte, donde la existencia de una clase depende de la otra.
 - vii. Ejemplo: Un Edificio tiene Habitaciones, si el edificio desaparece, las habitaciones también.
- ❖ Herencia (Generalización) (▲—)
 - viii. Representa una relación "es un".
 - ix. Ejemplo: Un Perro y un Gato heredan de Animal.
- ❖ Dependencia (∼)
 - x. Relación débil donde una clase usa a otra temporalmente.
 - xi. Ejemplo: Un Cliente usa un Servicio.
- ❖ Realización (⊢)
 - xii. Se usa cuando una clase implementa una interfaz.
 - xiii. Ejemplo: Una clase Carro implementa la interfaz Vehículo.

HERRAMIENTAS PARA HACER DIAGRAMAS DE CLASES

- ❖ Existen varias herramientas gratuitas que puedes utilizar para construir diagramas de clases UML. Algunas opciones populares incluyen:
- ❖ Draw.io (diagrams.net) - <https://app.diagrams.net/>
 - i. Basado en la web y de código abierto.
 - ii. Soporta UML y permite exportar en varios formatos.
- ❖ PlantUML - <https://plantuml.com/>
 - i. Basado en texto y permite generar diagramas automáticamente.
 - ii. Se puede integrar con VS Code, IntelliJ y otras herramientas.
- ❖ StarUML - <http://staruml.io/>
 - i. Herramienta de modelado UML con una versión gratuita limitada.
- ❖ Lucidchart (versión gratuita limitada) - <https://www.lucidchart.com/>
 - i. Plataforma en línea fácil de usar con colaboración en tiempo real.
- ❖ UMLet - <https://www.umlet.com/>
 - i. Herramienta de escritorio ligera y sencilla para diagramas UML.
- ❖ Modelio - <https://www.modelio.org/>
 - i. Software de modelado UML y BPMN gratuito con opciones avanzadas.

4. tipos de métodos en java

- ❖ Los métodos en Java son bloques de código que realizan tareas específicas. Tienen un nombre, pueden recibir parámetros como entrada y pueden devolver un valor con return. Dentro del método se pueden hacer cálculos, manipular datos o llamar a otros métodos.

Una ventaja clave es la reutilización de código, lo que evita repeticiones y mejora el mantenimiento del programa. Además, existe la recursión, donde un método se llama a sí mismo hasta alcanzar una condición de terminación.

En Java, podemos encontrar diferentes tipos de métodos según su estructura y funcionalidad:

- ❖ **Métodos sin retorno:** Estos métodos son aquellos que no devuelven ningún valor. Se utilizan principalmente para ejecutar un conjunto de instrucciones o modificar el estado de un objeto.

```
public void saludar() {  
    System.out.println("¡Hola, mundo!");  
}
```

- ❖ **Métodos con retorno:** Estos métodos devuelven un valor después de su ejecución. Este valor puede ser de cualquier tipo de dato válido en Java, como enteros, cadenas de texto, booleanos, entre otros. Por ejemplo, el resultado de una operación aritmética:

```
public int sumar(int a, int b) {  
    return a + b;  
}
```

- ❖ **Métodos estáticos:** Los métodos estáticos están asociados a la clase en lugar de una instancia de la clase. Estos métodos se pueden invocar directamente sin la necesidad de crear un objeto.

```
public static void mostrarMensaje() {  
    System.out.println("¡Hola, soy un ejemplo de método estático!");  
}
```

5. Los principios fundamentales de la programación orientada a objetos (POO) son:

- ❖ **Abstracción:**
 - i. Representa elementos del mundo real como objetos en un programa
- ❖ **Encapsulación:**
 - ii. Limita el acceso a los datos de los objetos
- ❖ **Herencia:**
 - iii. Permite reutilizar atributos y métodos comunes entre clases

- ❖ Polimorfismo:

- iv. Permite que los objetos tengan varios comportamientos

Estos principios permiten a los programadores desarrollar software de manera eficiente y estructurada.

Beneficios de la POO

- ❖ Reutilización de código
- ❖ Evita la duplicación de código
- ❖ Permite trabajar en equipo
- ❖ Protege la información
- ❖ Permite manejar la complejidad y variabilidad del software
- ❖ Permite adaptar los programas a diferentes necesidades y contextos

6. ¿que son las clases abstractas?

- ❖ Las clases abstractas en Java sirven como base para un grupo de clases, evitando la repetición de código mientras permiten que cada clase implemente sus propios detalles. Se definen con la palabra clave `abstract` y funcionan de manera similar a las clases concretas.

7. que son interfaces en java y como se construyen

- ❖ Las interfaces en Java actúan como plantillas que establecen un conjunto de reglas que las clases deben seguir. No pueden implementarse por sí mismas, sino que deben ser implementadas por otras clases. Definen métodos sin proporcionar su implementación, asegurando que cualquier clase que las implemente cumpla con un contrato de comportamiento.

Cómo se construyen las interfaces en Java

- ❖ En Java, una interfaz es una estructura que define un conjunto de métodos que una clase debe implementar. Se utilizan para establecer un contrato que las clases deben seguir, asegurando que todas las clases que implementen la interfaz tengan los mismos métodos.
- ❖ Para declarar una interfaz en Java, se usa la palabra clave `interface`. A diferencia de las clases, una interfaz no contiene implementaciones de métodos, solo sus firmas.

```
// Definición de una interfaz llamada Animal  
public interface Animal {  
    void hacerSonido(); // Método sin implementación  
    void mover();  
}
```

Conclusiones

La Programación Orientada a Objetos en Java proporciona una base sólida para desarrollar software estructurado y escalable. Su enfoque en clases y objetos mejora la organización del código, facilitando su reutilización y mantenimiento. Los diagramas de clases UML ayudan a visualizar la estructura del software antes de su implementación. La diversidad de métodos en Java permite mayor flexibilidad en la manipulación de datos y ejecución de operaciones. Además, la aplicación de los principios fundamentales de la POO permite construir sistemas más eficientes y modulares. Finalmente, las clases abstractas e interfaces en Java fomentan un diseño flexible y buenas prácticas de programación. Estos conocimientos permiten mejorar la calidad del software y adaptarse a diversas necesidades del desarrollo tecnológico.

Referencias Bibliograficas

<https://profile.es/blog/que-es-la-programacion-orientada-a-objetos/>

<https://www.netec.com/que-es-una-clase-en-java>

<https://openwebinars.net/blog/introduccion-a-java-metodos-parametros-y-argumentos/>

<https://www.santanderopenacademy.com/es/blog/que-es-una-interfaz-en-java.html>